Programming Assignment: CUDA Device Memory Analysis Assignment

You have not submitted. You must earn 80/100 points to pass.

Deadline Pass this assignment by Sep 4, 11:59 PM IST

Complete CUDA Device Memory Analysis

Instructions **My submissions** **Discussions**

Assignment Instructions
For this assignment, you will need to modify the host code and CUDA kernels to use device global, constant, shared, and register memory.

The host code will do the following steps:

1. Create an array of size elements per thread * number of threads to hold the search input data.
2. Place the data into the appropriate device memory based on the passed kernel value.
3. Start a timer for the kernel execution time.
4. Execute the kernel (based on using global, constant, shared, and register memory).
5. Stop the time for the kernel execution time.
6. Output the statistics for this iteration (partId, kernel, threads, elements per thread, kernel execution time in seconds) to output.cs

The general algorithm that each kernel (mostly isolating the use of each type of device memory) will be the following:

1. Determine the span of a threads search, which is basically the size of the input data set (a constant value) divided by the number of threads (also a constant value).
2. Increment the associated memory input value by 1.
3. The kernel will then iterate through the input data until it finds the expected value.
4. When it finds a value it will set the value in the input array for the associated index to 1 (the input array will start with all 0's)
5. It will complete searching through all of its assigned values and then the kernel will complete

Note that in the run.sh only global, shared, and register kernels are run in the line below:

PART_ID=RUsi7
kernels=(global shared register)
threadsPerBlock=128

To get a 100% on the assignment, you will need to add constant to those values. There may exist an issue with running the constant kernel as designed, so if you make that change you may need to update not only the kernel but the memory allocation/kernel execution host code. **You will still pass if everything else works and you do not make this change.**

For this assignment all executables will take the following command line arguments:
-m elementsPerThread - the number of elements that a thread will search for a random value in.
-p currentPartId - the Coursera Part ID
-t threadsPerBlock - the number of threads to schedule for concurrent processing, from 32 to 1024
-k the kernel type - global, constant, shared, register

Note that the max data size will be 64KB, so with each random value being a 16 bit integers (2 bytes), the max number of input values is 32K but to be safe we will leave it at 256 (max elements per thread) * 64 (threads per block), which is 16K. If the combination of threadsPerBlock and maxElementsPerThread are greater than 16K, then the threads per block will be kept and max elements per thread will be calculated by dividiing 16K by threads per block. The program will only run on one block, so that mean times do not include time to reschedule blocks of threads.

DO NOT TOUCH THE EXISTING PRINT STATEMENTS, THIS MAY CAUSE YOU TO FAIL AN ASSIGNMENT.
You can add debug print statement but it should at the end look like the following:

partId: partID elements: elementsPerThread threads: threadsPerBlock kernel: kernel